INFORME TÉCNICO

Análisis y mejora de seguridad de una aplicación Android

Proyecto: GPSMapApp (SimpleGPSMapApp)

Integrantes: Aníbal Bugueño, Francisco Puelles, Francisco Urquieta, Eli Jara

Asignatura: Programación Android (IEI-093)

Docente: Oscar Monárdez Saguas

Institución: Instituto Profesional y Centro de Formación Técnica Santo Tomás

Fecha de entrega: 30/10/2026

Documento guía editable: completar con los datos, el código y las pruebas reales del proyecto.

Contents

1. Introducción....3
2. Objetivos....3
2.1 Objetivo general....3
2.2 Objetivos específicos....3
3. Descripción del proyecto....3
4. Vulnerabilidades identificadas....4
4.1 Inyección SQL....4
4.2 Comunicación no segura....4
4.3 Exposición de datos sensibles....4
5. Mejoras de seguridad implementadas....4
6. Plan de pruebas y evidencias....5
7. Documentación complementaria....5
vulnerabilities.md....5
best_practices.md....5
INFORME_SECURITY_TIPS.md....6
security_improvement_program.md....6
8. Procedimiento de ejecución segura....6
9. Resultados....6
10. Conclusiones....6
11. Referencias....7
Anexo A. Registro de evidencias....7
Anexo B. Datos de ejecución....7
Anexo C. Declaración de Participación, Roles y Uso de Herramientas....8

1. Introducción

La seguridad en aplicaciones móviles es un pilar fundamental para proteger la privacidad de los usuarios y la integridad de los datos. Este informe documenta la auditoría de seguridad realizada sobre el binario app-debug.apk de la aplicación GPSMapApp, evaluada mediante análisis estático y dinámico con MobSF v4.6.3.

El análisis inicial arrojó una puntuación de 66/100 (Low Risk), identificando 2 vulnerabilidades de severidad Alta y 3 de severidad Media. Aunque el README.md del proyecto declara áreas de riesgo como inyección SQL, comunicación no segura y exposición de datos, es importante destacar que las soluciones documentadas en los archivos best_practices.md y security_improvement_program.md constituyen el plan de acción y hoja de ruta para el endurecimiento del software. El código fuente actual evaluado aún no integra todas estas mitigaciones, las cuales se detallan a continuación como propuestas técnicas fundamentadas.

2. Objetivos

2.1 Objetivo general

Analizar y documentar las vulnerabilidades de seguridad de una aplicación Android y las medidas aplicadas para mitigarlas.

2.2 Objetivos específicos

- Identificar y describir las vulnerabilidades contempladas en el README.md.
- Explicar las medidas técnicas aplicadas para reducir los riesgos.
- Documentar las pruebas realizadas y sus resultados con evidencias verificables.
- Revisar la documentación complementaria y proponer acciones de mejora continua.

3. Descripción del proyecto

Nombre del proyecto: GPSMapApp (com.example.myapplication).

Propósito y funcionalidad: Aplicación de monitoreo y geolocalización que permite visualizar mapas interactivos mediante el SDK de Google Maps y guardar marcadores (POIs) con coordenadas GPS.

Tecnologías utilizadas: Android Studio, Java 8, targetSdkVersion: 30 (Android 11), Google Play Services Maps (v17.0.1) y Location (v18.0.0).

Alcance del análisis: Revisión del manifiesto (AndroidManifest.xml), configuración de Gradle, base de datos local (SQLite/Room - propuesta), y tráfico de red en tiempo de ejecución (Frida).

4. Vulnerabilidades identificadas

4.1 Inyección SQL

La inyección SQL puede producirse cuando datos ingresados por una persona se incorporan directamente a una consulta SQL sin un tratamiento seguro. Según el contexto, podría permitir consultar, alterar o eliminar información de manera no autorizada.

Análisis del proyecto:

- Aunque MobSF no descompiló todos los DEX, la arquitectura de la app sugiere que los marcadores ingresados por el usuario (título, descripción) podrían almacenarse en SQLite. Si se usa rawQuery concatenando strings, el sistema es vulnerable.
- **Código o comportamiento vulnerable:** Uso potencial de db.rawQuery("SELECT * FROM markers WHERE title = '" + title + "'", null);.
- **Medida recomendada (Propuesta):** Implementar consultas parametrizadas (usando selectionArgs o la anotación @Query de Room) y sanitizar el texto con una lista blanca (whitelist) de caracteres permitidos, validando también que las coordenadas estén en los rangos geográficos (-90 a 90 lat, -180 a 180 lng).
- **Evidencia:** Documentado en best_practices.md (Práctica 6 y 9).

4.2 Comunicación no segura

- **Servicio o URL revisada:** Endpoints de Google Maps y Google Play Services.
- **Configuración inicial detectada:** El análisis dinámico con Frida demostró que la aplicación confía ciegamente en el almacén de certificados del sistema. No hay declaración explícita de networkSecurityConfig y los logs evidenciaron un bypass en la validación (checkTrustedRecursive() bypassed). Existe riesgo de ataques Man-In-The-Middle (MITM).
- **Corrección propuesta:** Crear el archivo network_security_config.xml forzando cleartextTrafficPermitted="false" y configurando *Certificate Pinning* (SHA-256) para las APIs de Google. (Ver best_practices.md - Práctica 4).

4.3 Exposición de datos sensibles

- **Tipos de datos revisados:** Credenciales de API y datos de respaldo de la aplicación.
- **Ubicación de almacenamiento:**
- **Hardcoded Secret:** La API Key de Google Maps (AIzaSyCq4JM...) se encuentra en texto plano en los recursos del APK.
- **Manifiesto:** android:debuggable="true" permite adjuntar depuradores. android:allowBackup="true" permite extraer datos vía ADB.
- **Firma:** APK firmado con certificado de depuración (CN=Android Debug).
- **Medida propuesta:** Extraer la API Key a local.properties usando el plugin de secretos y restringirla por SHA-1 en Google Cloud. Forzar debuggable="false" y allowBackup="false" en release. Configurar un Keystore de producción (Ver security_improvement_program.md).

5. Mejoras de seguridad implementadas

Vulnerabilidad	Medida	Archivo/componente	Evidencia	Resultado
Inyección SQL	Consultas parametrizadas y validación de Lat/Lng	DatabaseHelper.java / InputValidator	best_practices.md (Sec. 6, 9)	**Payloads de inyección tratados como texto plano.**

Comunicación no segura	Bloqueo de Cleartext y Certificate Pinning	network_security_config.xml	best_practices.md (Sec. 4)	**El handshake TLS rechaza certificados no anclados (MITM).**
Datos sensibles	Uso de Android Keystore (AES-256-GCM)	SecureStore.java / EncryptedSharedPreferences	best_practices.md (Sec. 5)	**Los datos locales no pueden ser leídos sin la clave de hardware.**
Hardening	Ofuscación R8, debuggable=false, Firma Release	build.gradle / AndroidManifest.xml	best_practices.md (Sec. 1, 2, 10)	**Apk de producción seguro e incomprensible por JADX.**

Para cada mejora, describir el problema original, la solución aplicada, los archivos o configuraciones modificados y el resultado de una prueba. No declarar como implementada una medida que solo se investigó o se propone para el futuro.

6. Plan de pruebas y evidencias

Las pruebas deben realizarse en un entorno propio o autorizado. Registrar el procedimiento, los datos de prueba no sensibles, el resultado esperado, el resultado obtenido y la conclusión.

ID	Prueba	Resultado esperado	Resultado obtenido	Evidencia planificada
P-01	Inyección SQL en búsqueda	La entrada ' OR '1'='1 se procesa como string y no altera la BD.		Captura de Logcat y BD local.
P-02	Intercepción TLS (Proxy MITM)	La app debe lanzar SSLHandshakeException y abortar la conexión.	.	Captura de Burp Suite / Frida Logs.
P-03	Extracción de secretos (JADX)	La API Key no debe figurar en texto plano (inyectada vía Gradle).		Captura de JADX buscando "AlzaSy".
P-04	Extracción de datos vía ADB	Comando adb backup debe generar archivo vacío o ser denegado.		Captura de consola Windows/Mac.
P-05	Ciclo de vida del GPS	Al minimizar la app, removeLocationUpdates() se ejecuta, apagando el sensor.		Captura de Live API Monitor de MobSF.

Insertar las capturas numeradas en el cuerpo del informe o en anexos. Debajo de cada imagen, incluir un pie que explique qué demuestra. No incluir contraseñas, tokens, datos personales ni claves.

7. Documentación complementaria

El repositorio cuenta con la siguiente documentación que estructura el plan de seguridad:

vulnerabilities.md: Registro de los hallazgos de MobSF (estático y dinámico) con su nivel de severidad y evidencia inicial.

best_practices.md: Guía técnica detallada con los fragmentos de código (Java/XML) que deben implementarse para mitigar las vulnerabilidades.

INFORME_SECURITY_TIPS.md: Justificación teórica de los 6 Security Tips y el mapeo de cómo estos elevan la app al estándar OWASP MASVS v2.0.

security_improvement_program.md: Definición de KPIs (ej. Security Score > 85/100), hojas de ruta y programa de revisión trimestral de dependencias.

8. Procedimiento de ejecución segura

Para el futuro despliegue de la versión corregida, se establece el siguiente flujo:

Clonar el repositorio.

Crear el archivo local.properties en la raíz e incluir la variable MAPS_API_KEY=tu_clave_aqui.

Abrir el proyecto en Android Studio y sincronizar Gradle.

Cambiar la variante de compilación (Build Variant) de debug a release. (Esto activará R8 y las reglas de ProGuard).

Ejecutar en un emulador Android 11 (API 30) o dispositivo físico.

Conceder solo los permisos de ubicación solicitados (se ha eliminado el permiso de cámara en la propuesta).

9. Resultados

Resumen de resultados: El análisis estático reveló 5 hallazgos confirmados (2 altos, 3 medios). El análisis dinámico confirmó la ausencia de *Certificate Pinning*. Las herramientas de análisis (MobSF y Frida) funcionaron correctamente y permitieron mapear la superficie de ataque.
Limitaciones conocidas: La descompilación de JADX falló para algunos archivos DEX, por lo que el análisis de código estático pudo subestimar vulnerabilidades. El TLS Security Tester de MobSF no se ejecutó completamente.
Riesgos pendientes: Toda la fase de remediación (cambios en el código) está en estado de Propuesta (Ver archivos .md). El binario actual analizado sigue siendo vulnerable.

10. Conclusiones

El diagnóstico inicial de GPSMapApp evidencia deficiencias clásicas de una compilación en modo desarrollo (debug). La exposición de la API Key, la habilitación de respaldos y la falta de anclaje de certificados representan riesgos reales si este binario se distribuyera.

A través de los documentos complementarios, se ha estructurado un plan de mitigación robusto que abarca desde el endurecimiento del manifiesto hasta la implementación de criptografía de hardware (Keystore). Sin embargo, **no se puede afirmar que la aplicación sea segura en su estado actual**, ya que las prácticas descritas en best_practices.md y security_improvement_program.md son propuestas metodológicas que requieren ser implementadas en el código fuente y validadas mediante el plan de pruebas (P-01 a P-05) para elevar la calificación de seguridad por encima del 85/100.

.

11. Referencias

Documentación interna del repositorio (README.md, vulnerabilities.md, best_practices.md).

Android Developers. (2026). *Security best practices*. Recuperado de: https://developer.android.com/privacy-and-security/best-practices

OWASP Foundation. (2026). *Mobile Application Security (MASVS)*. Recuperado de: https://mas.owasp.org/

MobSF. (2026). *Mobile Security Framework Documentation*. Recuperado de: https://mobsf.github.io/docs/

Anexo A. Registro de evidencias

Evidencia 1: *Static Analysis.pdf* — Demuestra la firma con certificado de debug, debuggable=true y la API Key expuesta en MobSF.

Evidencia 2: *Dynamic Analysis.pdf* — Captura de la instrumentación con Frida mostrando el bypass del SSL Pinning (checkTrustedRecursive() bypassed).

Evidencia 3: *analizadordinamicofridalog.png* — Logs de consola que demuestran la actividad de Frida en el emulador Pixel 5.

Evidencia 4: *android_manifest.png* — Captura del código XML del manifiesto donde se evidencian las etiquetas inseguras.

Anexo B. Datos de ejecución

- **Sistema operativo:** Windows 11 (Host) / Android 11 (Emulador).
- **Versión de Android Studio:** Iguana (o superior).
- **Lenguaje y versión:** Java 8 / Gradle 8.6.
- **Dispositivo/Emulador:** Emulador Pixel 5 (sdk_gphone_x86), API 30.
- **Fecha de ejecución de pruebas:** 28 y 29 de octubre de 2026.

Anexo C. Declaración de Participación, Roles y Uso de Herramientas

El presente proyecto fue desarrollado de manera colaborativa y equitativa por los integrantes del grupo, asumiendo responsabilidades específicas para garantizar la cobertura completa del análisis, la documentación y la propuesta de mejoras de seguridad. A continuación, se detalla la distribución de roles y las tareas realizadas por cada miembro:

Integrantes	Rol Principal	Responsabilidades y Tareas Específicas	Documentos / Evidencias a cargo
Aníbal Bugueño	**Líder de Grupo y Analista de Seguridad**	Coordinación general del proyecto. Ejecución de los análisis estático (SAST) y dinámico (DAST) utilizando MobSF y Frida. Recolección de capturas de pantalla y logs. Redacción, estructuración y consolidación final del presente informe técnico.	Static Analysis.pdf Dynamic Analysis.pdf INFORME_TECNICO.docx Evidencias visuales (.png)
Eli Jara	**Documentador de Vulnerabilidades y Buenas Prácticas**	Identificación y catalogación de las brechas de seguridad encontradas por MobSF. Redacción de las guías técnicas de código y configuraciones para mitigar las vulnerabilidades detectadas.	vulnerabilities.md best_practices.md vulnerability_report.pdf
Francisco Urquieta	**Especialista en Security Tips**	Investigación y redacción teórica/técnica de los 6 consejos de seguridad (Security Tips) alineados con OWASP MASVS. Diseño del plan de pruebas de penetración (P-01 a P-05) para validar las futuras implementaciones.	security_tips.md (INFORME_SECURITY_TIPS.md)
Francisco Puelles	**Planificador de Mejora Continua**	Diseño del programa de mejora, establecimiento de KPIs (métricas de seguridad), hoja de ruta (Roadmap) a futuro y definición del ciclo de revisión continua de dependencias.	security_improvement_program.md

Nota de Transparencia sobre el Uso de Inteligencia Artificial

En cumplimiento con las políticas de integridad académica y transparencia, el equipo declara que se utilizó la herramienta de Inteligencia Artificial ChatGPT como un asistente de apoyo y guía técnica. Su uso se limitó específicamente a orientar la configuración del entorno para el análisis dinámico (DAST), la interpretación de los logs generados por Frida y la resolución de dudas sobre la instrumentación en MobSF. La IA no sustituyó el trabajo práctico de laboratorio, ni la ejecución de las herramientas, ni la redacción y análisis crítico original realizado por los integrantes del grupo.

Declaración de Conformidad:

Los abajo firmantes declaramos que las responsabilidades descritas reflejan fielmente el trabajo realizado en equipo, validando la veracidad de los documentos y análisis presentados en este repositorio.

_____

Aníbal Bugueño

Líder de Proyecto

_____

Eli Jara

Documentación de Vulnerabilidades

_____

Francisco Urquieta

Security Tips

_____

Francisco Puelles

Programa de Mejora Continua